Task 1: SPPG mealbox delivery simulation

1. Problem and graph representation

The goal of this project is to simulate an SPPG delivery truck delivering mealboxes to schools in Surabaya within a given radius, starting and ending at the SPPG depot.

We represent the road network as a directed graph (digraph) G=(V,E). The vertices V are the road intersections, and the directed edges E are the streets. We must use a directed graph because the city contains one-way streets. If we used an undirected graph, the route could send the driver going against traffic. The weight of each edge is the physical road distance in kilometers calculated using the Haversine formula on GPS coordinates.

Since the SPPG and the schools are just points on the map and not necessarily exact street intersections, we project each facility to the closest road intersection in our graph.

2. Algorithms and how they work

A. Reachability check (double Dijkstra)

In a directed graph, finding a path to a school does not mean we can come back. If an intersection is inside a one-way street leading to a dead end, the vehicle gets stuck.

To solve this, we check reachability in both directions:

1. We run Dijkstra from the SPPG on the original graph G to find all nodes reachable from the SPPG.
2. We create the reversed graph G^T by flipping the direction of every edge (u,v) to (v,u), and run Dijkstra from the SPPG again. Reaching a node in G^T means that in the real world, that node can drive back to the SPPG.
3. We take the intersection of both node sets. This guarantees that all selected schools can both be reached from the SPPG and drive back to it.

B. Shortest path with dijkstra

To find the shortest road distance and the actual path between locations, we use Dijkstra's algorithm.

How it works:

1. We set the distance to the starting node to 0 and all other nodes to infinity. We push (0, start_node) into a min-heap priority queue (heapq) and initialize a parent dictionary.
2. In each step, we pop the node u with the smallest distance from the heap. If the distance popped is larger than what we already recorded, we skip it.
3. For each neighbor v of u, we check if going through u is shorter: new_dist = dist[u] + weight. If it is shorter, we update dist[v], save parents[v] = u, and push (new_dist, v) into the heap.
4. When the heap is empty, we have the shortest distances to all reachable intersections. We reconstruct the path to any destination by backtracking through the parents dictionary from the end to the start.

We run this from each delivery stop (the SPPG and each selected school) to build the complete K∗K distance matrix.

C. TSP Tour with Nearest Neighbor Heuristic

Finding the exact optimal solution for the Traveling Salesperson Problem is NP-hard. Checking all permutations would take (K−1)! steps, which freezes the app. We use the Greedy Nearest Neighbor heuristic instead.

How it works:

1. We start at the SPPG depot (index 0).
2. From the current location, we look at all schools not visited yet and pick the one with the smallest distance in our cost matrix.
3. We move to that school, add it to our tour list, remove it from the unvisited set, and repeat until all schools are visited.
4. Finally, we add the SPPG depot back to the end of the tour list to close the loop.

To draw the final green route on the map, we take each step of the tour and look up the saved list of street intersections from our Dijkstra runs, converting them into GPS coordinates.

3. Complexity Analysis

Let **V** be the number of road intersections, **E** the number of road segments, and **K** the number of delivery points (SPPG + selected schools).

Time Complexity

- **Dijkstra:** Running Dijkstra with a binary heap takes O((V+E)log(V)) as seen in class. We run it twice for the reachability check, and then K times to compute the shortest paths between all stops. This gives a total of O(K(V+E)log(V)).
- **Nearest Neighbor (TSP):** For K points, the loop runs K−1 times and looks through the remaining unvisited schools at each step. This takes O(K^2) comparisons.
- **Total Time:** The total time is O(K(V+E)log(V)+K^2). Since K is small (around 5 to 20 schools), the Dijkstra calls take almost all the computation time, while the TSP loop runs instantly.

Space Complexity

- **Road Graph:** Storing the network as an adjacency list takes O(V+E).
- **Dijkstra Structures:** The distance map, parent pointers, and heap take O(V) space per run.
- **Cost Matrix:** Storing the distance table between all K targets takes O(K^2) memory.
- **Total Space:** Overall, the program uses O(V+E+K^2) space.